

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: EC2 Instance Needs to Access an S3 Bucket

Scenario

Your application runs on an EC2 instance and needs to upload logs to an S3 bucket. A developer suggests storing AWS Access Keys inside the application.

Would you approve this? If not, what is the correct approach?

Solution

Never store Access Keys inside EC2 instances.

Use an **IAM Role** attached to the EC2 instance.

Architecture

```
Application
  |
EC2 Instance
  |
IAM Role
  |
Temporary Credentials (STS)
  |
Amazon S3
```

Steps

1. Create an IAM Role.
2. Attach the required S3 permissions.
3. Attach the role to the EC2 instance.
4. AWS automatically provides temporary credentials through the Instance Metadata Service (IMDS).

Sample IAM Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
```

```
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::company-logs/*"
    }
  ]
}
```

Why Roles Instead of Access Keys?

- Temporary credentials
- Automatic rotation
- No secrets stored on the server
- More secure

Scenario 2: EC2 in Account A Needs to Access an S3 Bucket in Account B

Scenario

Production EC2 is running in **Account A**, while the S3 bucket is in **Account B**.

How would you securely enable access?

Solution

Use **Cross-Account IAM Role**.

Architecture

Account A

EC2
|
IAM Role
|
AssumeRole
|

Account B

IAM Role
|
S3 Bucket

Step 1: Create IAM Role in Account B

Grant S3 permissions.

Example:

```
Action:
s3:GetObject
```

Step 2: Trust Policy

Allow Account A to assume the role.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111111111111:root"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Step 3: EC2 Role

Allow

```
sts:AssumeRole
```

on the Account B role.

Flow

EC2

↓

IAM Role

↓

STS AssumeRole

↓

Temporary Credentials

↓

Interview Keywords

- STS
 - AssumeRole
 - Trust Policy
 - Temporary Credentials
-

Scenario 3: Jenkins in One AWS Account Deploys to Another AWS Account

Scenario

Jenkins runs in a **Tools Account**.

Applications must be deployed into a **Production Account**.

How would you configure this securely?

Solution

Architecture

Tools Account

Jenkins

↓

IAM Role

↓

AssumeRole

↓

Production Account

↓

Deploy ECS/EKS/EC2

Steps

1. Jenkins EC2 uses an IAM Role.
2. Production Account creates a Deployment Role.
3. Trust policy allows the Jenkins Role to assume it.
4. Jenkins executes:

```
aws sts assume-role \  
--role-arn arn:aws:iam::PROD_ACCOUNT:role/DeploymentRole \  
--role-session-name JenkinsDeploy
```

5. Use the temporary credentials for deployment.
-

Advantages

- No production access keys
 - Temporary credentials
 - Centralized CI/CD
 - Least privilege
-

Scenario 4: Lambda Function Needs Access to DynamoDB and Secrets Manager

Scenario

A Lambda function reads configuration from Secrets Manager and stores data in DynamoDB.

How would you grant permissions?

Solution

Create a **Lambda Execution Role**.

Policy:

Allow:

secretsmanager:GetSecretValue

dynamodb:PutItem

logs:CreateLogGroup

```
logs:CreateLogStream
```

```
logs:PutLogEvents
```

Attach the role to the Lambda function.

Principle

Grant **only** the permissions required.

Avoid:

```
AdministratorAccess
```

Scenario 5: ECS Task Needs to Access an S3 Bucket

Scenario

An ECS application uploads invoices to S3.

Developers want to pass AWS keys as environment variables.

How would you implement this?

Solution

Use an **ECS Task Role**.

Architecture

Container

↓

Task Role

↓

Temporary Credentials

↓

Amazon S3

Policy

Allow

s3:PutObject

s3:GetObject

Difference

Task Execution Role

- Pull image from ECR
- Write logs

Task Role

- Application permissions

This distinction is asked frequently in interviews.

Scenario 6: IAM User Has Full S3 Access but Still Gets Access Denied

Scenario

A developer has the following IAM policy:

```
AmazonS3FullAccess
```

Yet access is denied.

How would you troubleshoot?

Solution

Check in this order:

1. IAM Policy

Does it allow the action?

2. Bucket Policy

Is there an explicit Deny?

3. Service Control Policy (AWS Organizations)

Could an SCP be blocking access?

4. Permission Boundary

Is it restricting permissions?

5. Session Policy

If using AssumeRole, is the session policy limiting access?

6. Resource Encryption

Is the bucket using a KMS key that also requires permissions?

AWS Evaluation Logic

Explicit Deny

↓

Overrides

↓

Allow

Interview Tip

Always mention:

"I would check for Explicit Deny first because it overrides all Allow permissions."

Scenario 7: Temporary Access Required for an External Vendor

Scenario

A third-party vendor needs access to one S3 bucket for one week.

How would you provide access?

Solution

Create a dedicated IAM Role.

Trust Policy

Allow the vendor's AWS Account to assume the role.

Grant only:

`s3:GetObject`

Bucket:

`vendor-reports`

Optionally:

- Use an **External ID** to prevent the confused deputy problem.
 - Remove or disable the role after one week.
-

Benefits

- No permanent credentials
 - Easy to revoke
 - Least privilege
-

Scenario 8: Multiple AWS Accounts Need Centralized Administration

Scenario

Your organization has:

- Dev
- Test
- UAT
- Production

Administrators should log in once and access all accounts securely.

Solution

Use:

- AWS Organizations
- IAM Identity Center (formerly AWS SSO)
- Cross-Account Roles

Architecture

IAM Identity Center

↓

Developer Login

↓

Dev

↓

Test

↓

UAT

↓

Production

Each account contains an IAM role that Identity Center users can assume based on assigned permission sets.

Benefits

- Centralized authentication
- No individual IAM users in each account
- Simplified access management
- MFA support

Scenario 9: Application Running on EKS Needs Access to S3

Scenario

Pods need to upload files to S3.

Developers want to store AWS credentials inside Kubernetes Secrets.

Would you approve?

Solution

No.

Use **IAM Roles for Service Accounts (IRSA)**.

Architecture

Pod

↓

Service Account

↓

IAM Role

↓

STS

↓

Amazon S3

Benefits

- No AWS keys inside Pods
- Temporary credentials

- Least privilege
 - Fine-grained permissions per workload
-

Interview Tip

Mention:

EC2 IAM Roles provide permissions at the node level, while **IRSA provides permissions at the Pod level**, which is more secure for multi-tenant clusters.

Scenario 10: Cross-Account Deployment Pipeline Failed

Scenario

Your Jenkins pipeline suddenly fails with:

```
AccessDenied
```

```
Not authorized to perform sts:AssumeRole
```

How would you troubleshoot?

Solution

Step 1

Verify the source role has permission:

```
sts:AssumeRole
```

Step 2

Check the destination role's **Trust Policy**.

Confirm it trusts the correct IAM role or AWS account.

Step 3

Verify the Role ARN.

Ensure the account ID, role name, and ARN are correct.

Step 4

Check SCPs.

AWS Organizations may be denying `sts:AssumeRole`.

Step 5

Review CloudTrail.

Look for:

`AssumeRole`

events to identify why access was denied.

Step 6

Check Session Duration.

If the session expired, obtain new temporary credentials.

Common Causes

- Incorrect trust relationship
 - Missing `sts:AssumeRole` permission
 - Wrong Role ARN
 - SCP restrictions
 - Permission boundaries
 - Session policy restrictions
 - Expired temporary credentials
-

IAM Roles & Cross-Account Interview Cheat Sheet

Component	Purpose
IAM User	Long-term identity for humans (generally avoid for applications)
IAM Role	Temporary permissions assumed by users, services, or applications
IAM Policy	Defines allowed or denied actions
Trust Policy	Specifies who can assume a role
STS (Security Token Service)	Issues temporary security credentials
AssumeRole	API used to obtain temporary credentials for another role
Instance Profile	Container that attaches an IAM role to an EC2 instance
EC2 IAM Role	Grants permissions to applications running on EC2
Lambda Execution Role	Grants Lambda access to AWS services
ECS Task Role	Grants application containers AWS permissions
ECS Task Execution Role	Used by ECS to pull images from ECR and send logs to CloudWatch
IRSA	Grants IAM permissions to individual Kubernetes service accounts in EKS
Cross-Account Role	Enables secure access between AWS accounts
External ID	Protects against the confused deputy problem in third-party access scenarios
SCP (Service Control Policy)	Organization-level permission guardrail that limits the maximum available permissions
Permission Boundary	Restricts the maximum permissions an IAM principal can receive
Resource-Based Policy	Attached directly to resources such as S3 buckets, KMS keys, or Lambda functions

Production Best Practices (Interview Highlights)

- **Never** store AWS Access Keys on EC2, ECS, Lambda, or EKS workloads.
- Use **IAM Roles** with **temporary STS credentials** whenever possible.
- Follow the **Principle of Least Privilege**.
- Use **cross-account AssumeRole** instead of sharing credentials.
- Protect third-party access with **External IDs**.
- Enable **MFA** for privileged users and administrative access.
- Use **CloudTrail** to audit IAM and STS activity.
- Regularly review IAM policies with tools like **IAM Access Analyzer** and remove unused permissions.